

Smart Chat App



Session 2024 - 2028

Submitted By:

Muhammad Husnain

2024 CS 696

Seerat Fatima

2024 CS 663

Ayan Sajid

2024 CS 661

Submitted To:

Ms. Anum Iftikhar

Department of Computer Science, UET Lahore, New Campus



Table of Contents

1	Abstract	4
2	Introduction	5
2.1	Background	5
2.2	Problem Statement	5
2.3	Proposed Solution	5
2.4	Objectives	6
2.5	Scope	6
3	UML Diagrams	7
3.1	System Overview	7
3.2	Flow Charts	8
3.2.1	Login	8
3.2.2	Active Users	9
3.2.3	Send Message	10
3.2.4	Send File	11
3.2.5	Call	12
		12
3.3	System Architecture Diagram	13
3.4	System Architecture Diagram	14
3.5	System Architecture Diagram	15
3.6	System Architecture Diagram	16
4	Project Overview	16
4.1	Architectural Overview	16
4.1.1	Use of the Kurose & Ross Top-Down Approach	16
4.2	Architecture Layers (Based on the OSI Model – 5 Core Layers)	17
4.2.1	Application Layer	17
4.2.2	Transport Layer	17
4.2.3	Network Layer	18
4.2.4	Data Link Layer	18
4.2.5	Physical Layer	18
5	Conclusion	19



5.1	Project Summary.....	19
This project presents the design and implementation of a real-time multi-user chat application based on a client–server architecture using Python socket programming. The system enables real-time group communication through chat rooms, private one-to-one messaging, file sharing, and voice/video calling. A custom application-layer protocol was developed to support structured and secure communication, while multi-threading was used to ensure efficient handling of concurrent clients. The project demonstrates the practical application of core Computer Networks concepts		
5.2	Objectives Achievement	19
5.3	Technical Achievements	19
5.4	Challenges and Solutions.....	20
5.5	Limitations	20
5.6	Future Enhancements.....	21
5.7	Learning Outcomes	21
5.8	Final Remarks	21

Table of Flowcharts

Flowchart 1 System Overview.....	7
Flowchart 2 Login.....	8
Flowchart 3 Active List.....	9
Flowchart 4 Send Message	10
Flowchart 5 Send File	11

Table of Architecture Diagrams

System Architecture Diagram 1	13
System Architecture Diagram 2	14
System Architecture Diagram 3	15
System Architecture Diagram 4	16



1 Abstract

This project focuses on the design and implementation of a **real-time multi-user chat application** using Python socket programming, developed as part of the Computer Networks coursework. The system follows a **client-server architecture** and enables multiple users to communicate simultaneously over a local network. Core functionalities include group chat through rooms, private messaging, file sharing, and real-time voice and video communication.

The project is designed using the **Kurose and Ross top-down approach**, where application requirements are defined first, followed by the selection of appropriate transport and network layer services. A custom application-layer protocol is implemented to support structured message exchange, message framing, encryption, and efficient routing. The server employs a multi-threaded model to handle concurrent client connections, while the client application provides a graphical user interface for improved usability.

This project demonstrates the practical application of key networking concepts such as TCP/IP communication, socket programming, multi-threading, protocol design, and real-time data transmission. The implementation successfully meets all defined objectives and serves as a strong educational example of modern networked application development.



2 Introduction

2.1 Background

With the rapid growth of the internet and digital communication technologies, real-time communication systems have become an essential part of modern life. Applications such as messaging platforms, video conferencing tools, and collaborative systems are now widely used in education, business, and social interaction. These systems rely heavily on computer networking principles such as client–server architecture, data transmission protocols, encryption, and concurrency handling.

2.2 Problem Statement

Despite the availability of many communication platforms, there is often a lack of understanding of how these systems work internally. Students usually learn networking concepts such as the OSI model, TCP/IP protocol stack, encryption, and client–server architecture in theory, but they rarely see how these concepts are applied in a complete working system.

Another major issue is that many existing chat systems are complex and not suitable for educational exploration. Their internal architectures are difficult to analyze, and their codebases are too large for beginners to understand. As a result, learners struggle to connect theoretical knowledge with real-world applications.

The problem addressed in this project is the absence of a simple yet comprehensive educational system that demonstrates real-time communication using layered architecture, secure transmission, and multi-client handling in a clear and understandable way.

2.3 Proposed Solution

To overcome these challenges, this project proposes the development of a **real-time multi-user chat application** based on a structured and layered design approach. The system follows a client–server architecture where multiple clients connect to a centralized server responsible for managing communication.

The solution focuses on applying networking theory in practice by implementing:

- A **layered communication model** separating presentation, application, protocol, security, and transport responsibilities.
- **Secure data transmission** using encryption to protect messages and files.
- **Reliable communication** using TCP sockets to ensure message delivery.
- **Concurrent client handling** through multithreading.
- Support for **text messaging, file sharing, and voice/video communication**.



Rather than focusing only on functionality, the project emphasizes architectural clarity so that each layer and process can be easily understood from a theoretical perspective.

2.4 Objectives

The main objectives of this project are:

1. To demonstrate the **practical implementation of networking concepts** such as client–server communication, layered architecture, and secure transmission.
2. To design a system that clearly shows **how messages flow through different layers** of a communication model.
3. To provide a **learning-oriented platform** for understanding real-time communication systems.
4. To implement a **modular architecture** that separates concerns and improves maintainability.
5. To analyze how **multithreading and concurrency** improve system performance.
6. To ensure that the system follows **reliability, security, and scalability principles**.
7. To create a project that can serve as a **reference model** for future networking and distributed system studies.

2.5 Scope

The scope of this project is focused on the **theoretical and architectural understanding** of real-time communication systems rather than commercial deployment. The system is designed mainly for educational and academic purposes.

The project includes:

- A complete **client–server communication framework**.
- Demonstration of **layered system design**.
- Implementation of **secure data transfer** mechanisms.
- Support for **text communication, file transfer, and multimedia signaling**.
- Handling of **multiple clients simultaneously**.

However, the project does not aim to:

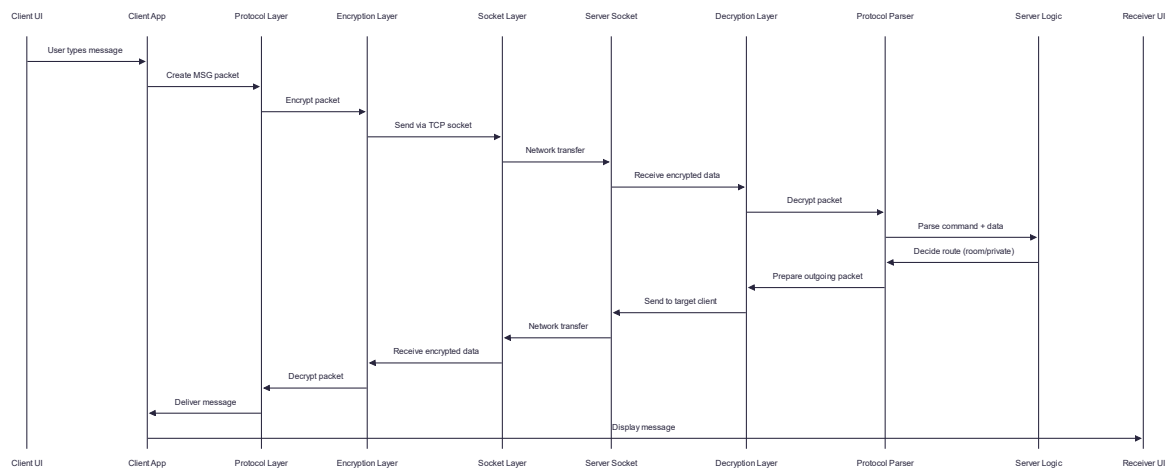
- Replace commercial chat platforms.
- Provide enterprise-level scalability or cloud deployment.
- Implement advanced authentication or identity management systems.



Instead, the scope is limited to building a **clear, understandable, and well-structured model system** that explains how real-time communication works from a theoretical and architectural perspective.

3 UML Diagrams

3.1 System Overview

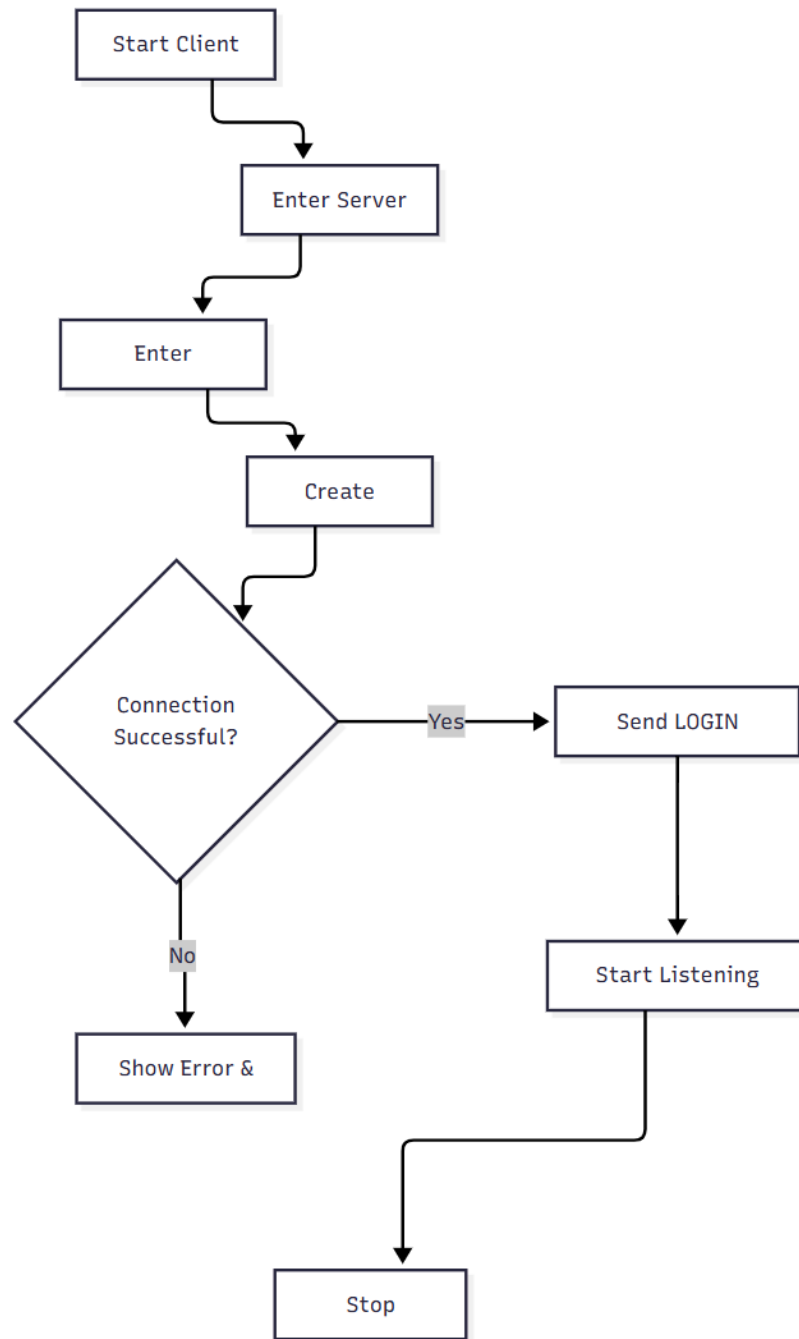


Flowchart 1 System Overview



3.2 Flow Charts

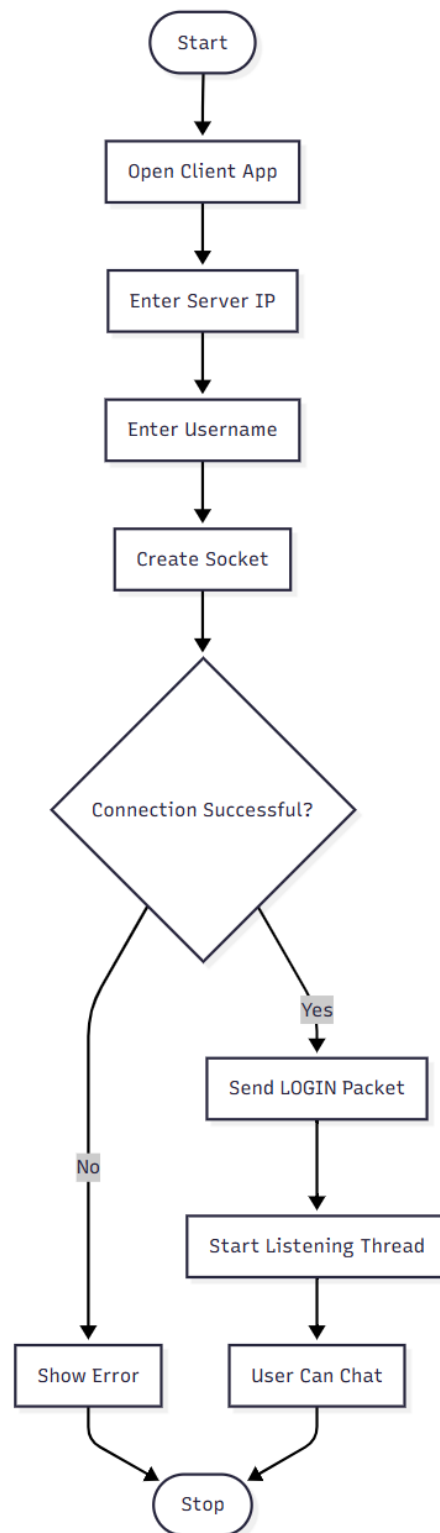
3.2.1 Login



Flowchart 2 Login



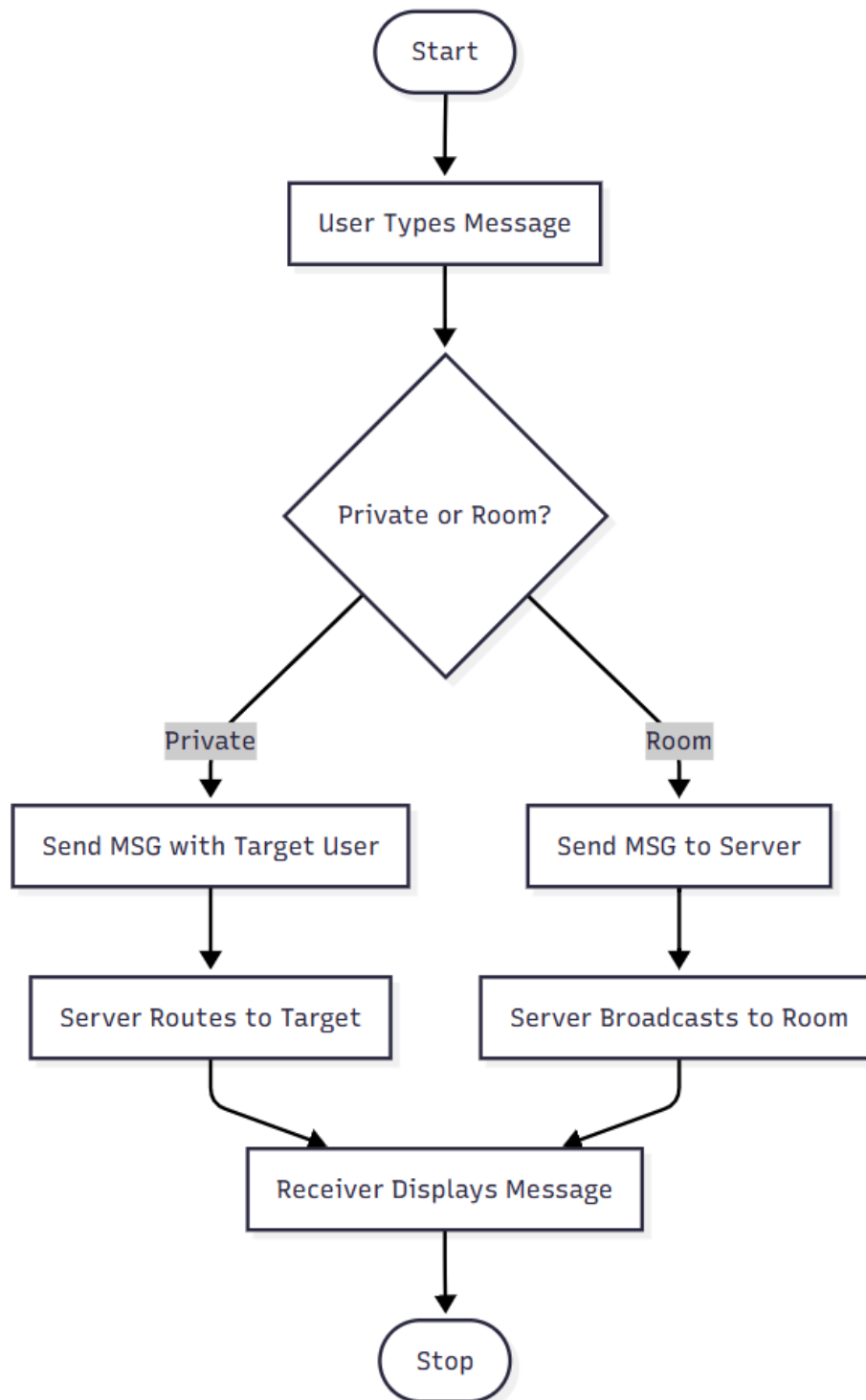
3.2.2 Active Users



Flowchart 3 Active List



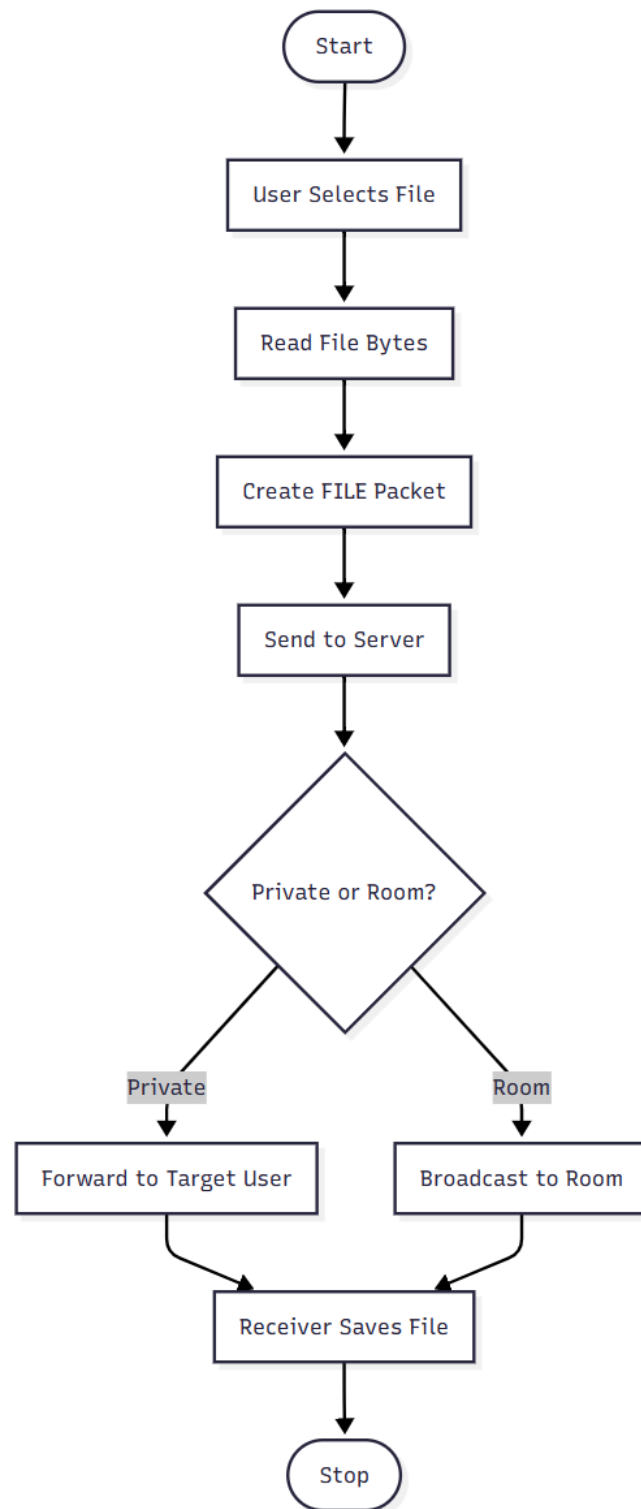
3.2.3 Send Message



Flowchart 4 Send Message



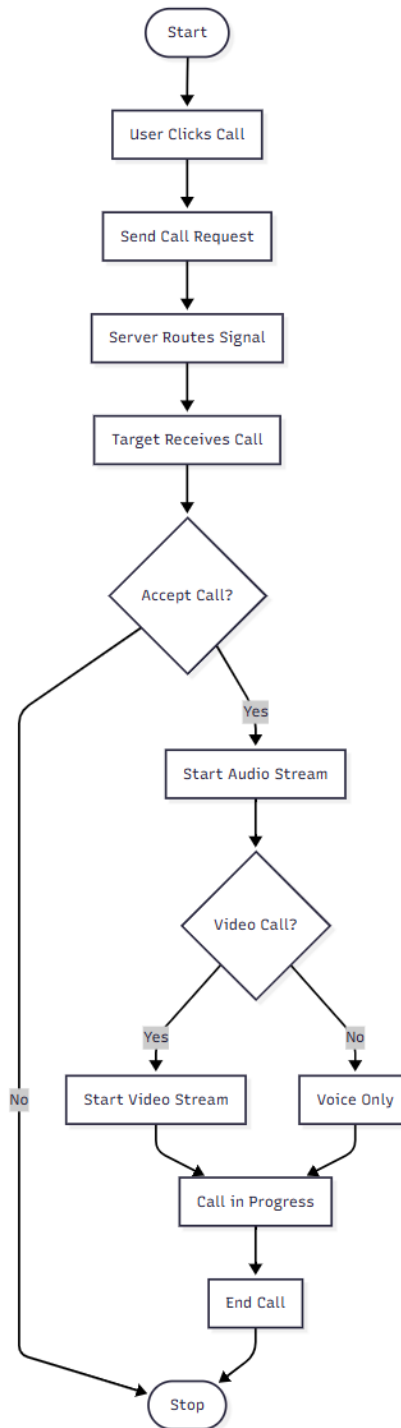
3.2.4 Send File



Flowchart 5 Send File



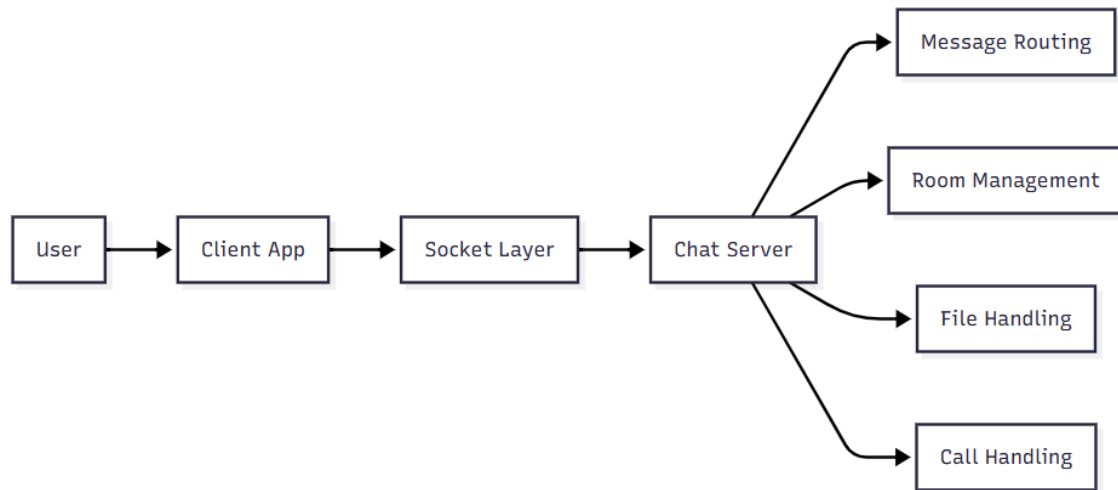
3.2.5 Call



Flow Chart : 6 Call



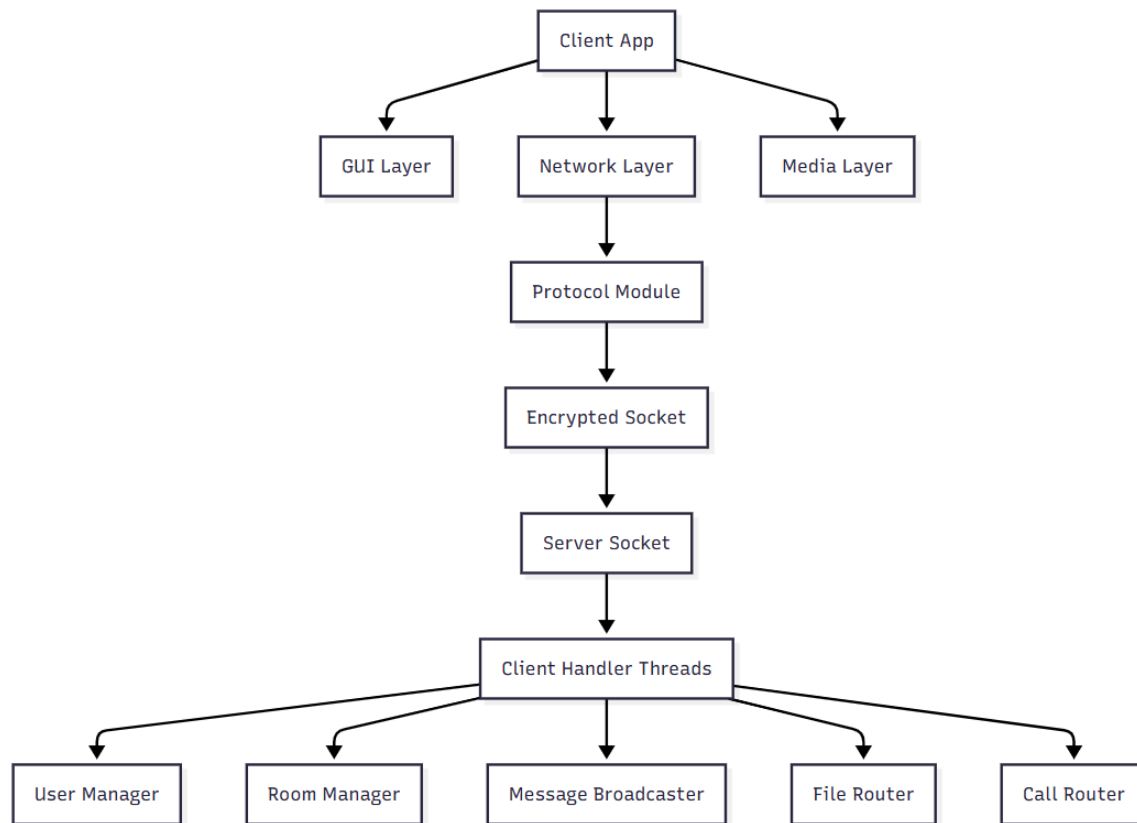
3.3 System Architecture Diagram



System Architecture Diagram 1



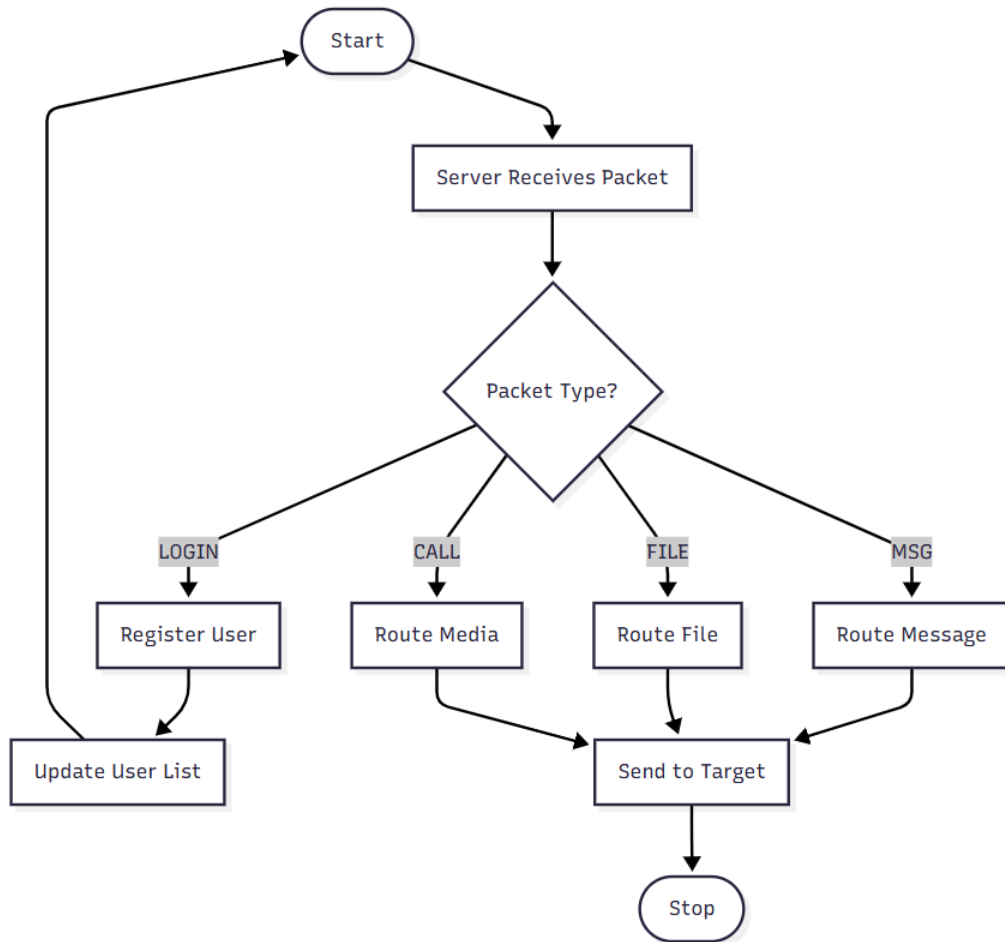
3.4 System Architecture Diagram



System Architecture Diagram 2



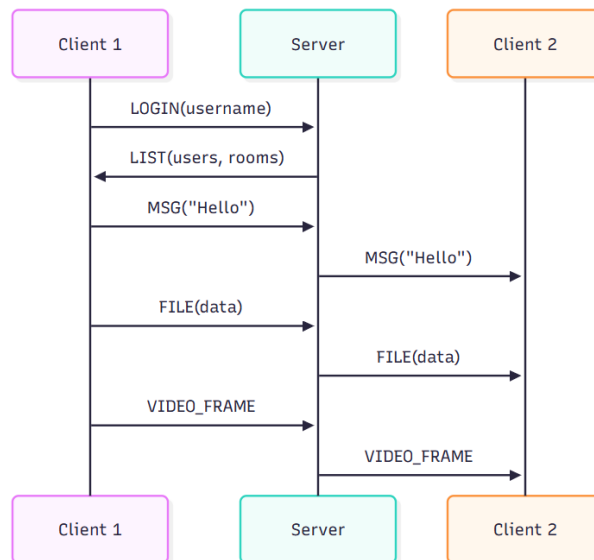
3.5 System Architecture Diagram



System Architecture Diagram 3



3.6 System Architecture Diagram



System Architecture Diagram 4

4 Project Overview

4.1 Architectural Overview

This system follows a **top-down layered approach**, where communication starts from the user level and moves downward through the network stack until it reaches the physical transmission medium. Each layer has a specific responsibility and works in coordination with the others to ensure reliable and efficient communication.

4.1.1 Use of the Kurose & Ross Top-Down Approach

In this project, the **Kurose and Ross Top-Down Networking Approach** was intentionally used to design and implement the chat application. This methodology starts from the **application layer** and progressively moves downward toward the **transport and network layers**, focusing first on *what the application needs to do* rather than how lower-level networking mechanisms work internally.

The design follows a **modular structure**, where each component is responsible for a specific task. This separation of responsibilities allows the system to be easily maintained and extended in the future. The architecture ensures that:



- The **client side** handles user interaction and media capture.
- The **server side** handles coordination, message delivery, and user management.
- The **network layer** ensures reliable data transfer between all components.

The system is not only functional but also educational, as it clearly demonstrates how theoretical networking concepts are applied in real-world communication systems.

4.2 Architecture Layers (Based on the OSI Model – 5 Core Layers)

4.2.1 Application Layer

The Application Layer is the **highest layer** and the closest to the user. It provides the interface between the user and the communication system. All user actions such as sending messages, sharing files, and starting calls originate from this layer.

- **Role in this project**
- Handles chat operations (send/receive messages)
- Manages users and rooms
- Controls file sharing and call requests
- Interprets user commands

This layer defines **what** the system does from the user's point of view.

4.2.2 Transport Layer

The Transport Layer is responsible for **end-to-end communication** between clients and the server. It ensures that data sent from one device reaches the other correctly.

- **Role in this project**
- Uses **TCP** for reliable communication
- Ensures ordered delivery of messages
- Handles retransmission in case of errors
- Manages flow control

This layer defines **how data is delivered reliably**.



4.2.3 Network Layer

The Network Layer manages the **logical transmission of data** across networks. It ensures that data packets reach the correct destination device.

- **Role in this project**
- Uses **IP addressing**
- Routes data between client and server
- Supports LAN and internet communication
- Handles packet forwarding

This layer defines **where the data goes**.

4.2.4 Data Link Layer

The Data Link Layer is responsible for **node-to-node delivery** of data within the same network. It ensures that frames are correctly transferred between directly connected devices.

- **Role in this project**
- Handles **MAC addressing**
- Detects transmission errors
- Controls access to the physical medium
- Ensures frame synchronization

This layer defines **how devices communicate on the same network**.

4.2.5 Physical Layer

The Physical Layer is the **lowest layer** and deals with the actual transmission of bits over a physical medium.

- **Role in this project**
- Transfers raw bits (0s and 1s)
- Uses cables, Wi-Fi signals, or fiber optics
- Defines voltage levels and signal timing
- Controls data transmission speed

This layer defines **how data physically moves**.



5 Conclusion

5.1 Project Summary

This project presents the design and implementation of a real-time multi-user chat application based on a client-server architecture using Python socket programming. The system enables real-time group communication through chat rooms, private one-to-one messaging, file sharing, and voice/video calling. A custom application-layer protocol was developed to support structured and secure communication, while multi-threading was used to ensure efficient handling of concurrent clients. The project demonstrates the practical application of core Computer Networks concepts

5.2 Objectives Achievement

The project successfully met all defined objectives. The implemented system fulfills the functional, technical, and performance requirements established at the beginning of development.

Achieved objectives include:

- Supporting multiple simultaneous client connections
- Real-time group and private messaging
- Dynamic room creation, joining, and management
- File transfer between users and within rooms
- Voice and video call functionality
- Active user and room list updates

All features were tested in a local network environment and performed as expected.

5.3 Technical Achievements

Several important technical milestones were achieved during the development of this project:

- Design and implementation of a **custom application-layer protocol**
- Reliable communication using **TCP sockets**
- **Multi-threaded server** for concurrent client handling
- Thread-safe data access using synchronization mechanisms
- Secure data transmission using **encryption**
- Real-time **audio and video streaming** using chunk-based transmission
- Modular and maintainable code structure with clear separation of concerns



These achievements reflect correct use of networking principles and good software engineering practices.

5.4 Challenges and Solutions

The development process involved several challenges, which were resolved through careful design and implementation:

Challenge	Solution
Handling multiple clients concurrently	Multi-threading with thread locks
TCP message boundary issues	Length-based message framing
UI freezing during network operations	Background threads for network communication
Media streaming latency	Chunk-based transmission with controlled frame rates
Resource synchronization	Centralized locking for shared data

Addressing these challenges significantly improved system stability and performance.

5.5 Limitations

Despite its successful implementation, the system has certain limitations:

- No persistent database for user accounts or message history
- Static encryption key limits security scalability
- Media quality depends on hardware and network conditions
- Not optimized for large-scale or internet-wide deployment

These limitations are acceptable given the academic scope of the project.



5.6 Future Enhancements

The project can be extended and improved in several ways:

- Integration of a database for persistent storage
- Implementation of secure authentication and dynamic key exchange
- Enhanced media compression and adaptive streaming
- Web-based or mobile client support
- Cloud deployment for global accessibility
- Improved graphical interface using modern frameworks

Such enhancements would significantly increase scalability and real-world applicability.

5.7 Learning Outcomes

This project provided valuable hands-on experience in both theoretical and practical aspects of computer networking.

Key learning outcomes include:

- Understanding of TCP/IP and socket programming
- Application of the **Kurose & Ross top-down approach**
- Design of custom application-layer protocols
- Multi-threading and concurrency control
- Real-time data and media communication
- GUI-based network application development
- Debugging and performance optimization techniques

5.8 Final Remarks

In conclusion, this project represents a complete and well-structured implementation of a real-time communication system. It effectively demonstrates the application of networking concepts, thoughtful architectural design, and disciplined software development practices. The project meets academic requirements while also providing a strong foundation for future work in advanced networking and distributed systems.